

Viscraft — Assessment Response

Software Engineer Assignment — Actual Inc (Indonesia) | Candidate submission documentation

Section 1: The Niche

Niche: Product Ad Photography for small businesses and entrepreneurs.

The niche was chosen deliberately, not arbitrarily. Small business owners frequently need professional product photos — for online stores, social media ads, and pitch decks — but a proper product photography session costs hundreds of dollars and requires scheduling, a photographer, and post-processing. Viscraft eliminates all of that.

This is meaningfully different from a generic AI art generator. The entire product is shaped around the constraints and expectations of commercial product photography:

- Prompts are structured around product, not artistic expression — background, lighting, mood, angle, and props are the relevant controls.
- The prompt-building system (`buildPrompt()`) outputs prompts in a format that AI image models are trained to interpret as product photography: *"Commercial product photography, high resolution, sharp focus on product, no watermark, no extra text."*
- The gallery is organized by campaign (product launch or brand context), not by a chronological dump of images.
- Regeneration is designed around iteration: tweak the prompt, keep the original as a reference, generate again — the same workflow a photographer uses when reviewing test shots.

The niche shaped every product decision, from naming (campaigns, ad shots) to the prompt options table schema (category, label, promptValue, sortOrder), to the default suffix appended to every generated prompt.

Section 2: App Journey (Request Flow)

A detailed trace of how a user's prompt travels from the browser through the backend to the AI API and back as a rendered image.

- 1 User fills the form in React.** `buildPrompt()` assembles the structured prompt from the user's product description and selected styling options in real-time. The user sees the assembled prompt in a preview pane before submitting.
- 2 Frontend calls POST /scenes/generate.** The Axios instance (with JWT interceptor) sends the request body:
`{projectId, prompt, generatedPrompt, referenceSceneId?, uploadedReferenceImage?}`
- 3 Gin handler validates the request.** JWTAuth middleware extracts `userId` from the token. The handler binds the JSON body and passes it to `SceneService.Generate()`.
- 4 Service validates and inserts.** Validates prompt length and blocked words, verifies the campaign belongs to the user, calculates the next `order_index`, then calls `repository.InsertProcessing()`.

```
// From service/sceneService.go
sceneId, err := s.repository.InsertProcessing(userId, req, orderIndex)
if err != nil {
    return response.GenerateSceneResponse{}, &constant.ErrDatabaseFailed
}

generatedPrompt := req.Prompt
if req.GeneratedPrompt != "" {
    generatedPrompt = req.GeneratedPrompt
}

go s.processGeneration(requestId, sceneId, generatedPrompt, referenceImageBytes)
```

- 5 HTTP 202 returned immediately.** The handler responds with the new scene ID and `status: "processing"` — before any AI call is made. The client now has a scene ID to poll against.
- 6 Background goroutine runs.** `processGeneration()` runs concurrently. It decides which Pollinations endpoint to call based on whether a reference image was provided, then sets a context deadline (60s for text-only, 90s with reference).

```
// From pkg/imagegen/client.go
func (c *Client) Generate(ctx context.Context, prompt string, referenceImage []byte) ([]byte, error) {
    if referenceImage == nil {
        return c.generateTextOnly(ctx, prompt)
    }
    return c.generateWithReference(ctx, prompt, referenceImage)
}

// Text-to-image: GET request
endpoint := pollinationsImageURL + "/" + url.PathEscape(prompt) + "?model=kontext&nologo=true"

// Image-editing: POST multipart
// Fields: prompt, model=kontext
// File: image (reference PNG)
```

- 7 On AI success:** goroutine saves the returned PNG bytes to `./storage/images/{sceneId}.png` and calls `repository.UpdateCompleted(sceneId, filePath, fileUrl)`.
- 8 On AI failure:** goroutine calls `repository.UpdateStatus(sceneId, "failed", errorCode)`. The error code is determined by the type of failure: context deadline exceeded → `ERR_03`, content policy string in error → `ERR_13`, all others → `ERR_05`.
- 9 Frontend polls via SWR.** After receiving the 202, SWR's `refreshInterval` callback fires `POST /scenes/get` every 3 seconds. While status is `"processing"`, a skeleton card is shown in the gallery grid.
- 10 Terminal state renders.** When the poll returns `status: "completed"`, SWR stops polling and the skeleton resolves into the final image card. On `status: "failed"`, an error card with the error code is shown instead.

Section 3: Tech Stack Choice & Justification

Every technology choice was made to serve the specific constraints of this problem: async image generation, multi-user isolation, persistent gallery, and handling a slow, unreliable AI API.

Technology	Reason for Choice
Go + Gin	Go's goroutines are the right primitive for async AI generation — each generate call spawns a goroutine, and Go's runtime handles thousands of these efficiently without threads. Gin adds minimal overhead and has clean JSON binding and middleware support. The standard library handles file I/O, HTTP, and base64 without any additional dependencies.
PostgreSQL	ACID compliance is necessary for concurrent user isolation. Multiple users generating simultaneously need guaranteed <code>order_index</code> values, correct cascade deletes, and row-level isolation. A document store or in-memory solution would require reimplementing guarantees that Postgres provides out of the box.
React 19 + TypeScript	TypeScript eliminates entire categories of runtime errors (wrong field names, missing nullish checks) that would be particularly painful when mapping between backend response shapes and UI state. React 19's concurrent features weren't specifically needed, but the ecosystem (Chakra UI, SWR, Zustand) is mature and well-suited to this UI pattern.
SWR	Built-in <code>refreshInterval</code> support makes polling trivially simple: pass a callback that returns an interval when status is <code>"processing"</code> and <code>0</code> (stop) otherwise. Without SWR, this would require manual <code>useEffect</code> timers with cleanup — more code, more bugs.
Zustand	Auth state needs to persist across page refreshes (localStorage) and be accessible from any component. Zustand's <code>persist</code> middleware handles this in ~5 lines. Redux would be overkill; React Context doesn't play well with localStorage persistence.
Chakra UI v3	Component library with a well-defined token system makes it practical to implement a custom design system (Cartographer's Atlas). The alternative — unstyled primitives + custom CSS — would have taken significantly longer for no meaningful benefit to the assignment.
Pollinations.ai (kontext model)	Free tier with an API key. Supports both text-to-image (GET) and image-editing (POST multipart) in a single API surface. The kontext model produces commercially usable product photography. The main tradeoff is reliability — image editing mode has a ~20% success rate for complex edits — but for the scope of this assignment the free tier and dual-mode support outweighed the reliability concerns.
Local filesystem storage	No CDN or object storage was set up. Images are written to <code>./storage/images/</code> and served by Go's static file handler (or nginx in production). This was a deliberate simplification — adding S3 would have introduced AWS credential management and IAM complexity without improving the core product for this submission.

Section 4: Build Process

The build was sequenced to de-risk the hardest parts first — specifically the AI integration and async flow — before investing in UI polish.

- Backend core first.** Implemented user auth (JWT), project CRUD, and the DB schema. Validated the complete request/response cycle with Postman before writing any frontend code.
- Scene generation with async flow.** Built the goroutine pattern, the polling endpoint, and the status state machine (processing → completed/failed) before building the frontend form. This is the highest-risk part of the system.
- Frontend auth + campaign management.** Built the landing page, auth modal, Zustand stores, and Axios JWT interceptor. Implemented the campaign sidebar.
- Scene gallery with polling.** Connected SWR polling to the backend generate endpoint. Got the end-to-end flow

working with a basic UI.

5. **Design system (Cartographer's Atlas).** Applied the Chakra UI theme — color tokens, typography, component variants. This was done after the functional flow worked to avoid wasted styling effort on components that might change.
6. **Pivot to product ad niche.** Renamed entities ("images" → "scenes/ad shots", "projects" → "campaigns"), added the `prompt_options` table and migration, built the `buildPrompt()` system, and updated the generate form with the options UI.
7. **UI improvements.** Storyboard-style grid layout, detail modal, card hover effects, skeleton loading states, regenerate flow.
8. **Error handling and failure states.** Implemented the three async failure modes (ERR_03, ERR_05, ERR_13), the rate-limit banner, and form validation error display.
9. **Supporting features.** Onboarding tour, sample campaign auto-seed, 404 page, daily log file rotation.
10. **Documentation.** API reference, tech stack writeup, application guide, and this assessment response.

Section 5: Key Technical Decisions

Decision 1: Async Generation via Goroutines

Problem: Pollinations API calls take 10–90 seconds. Holding an HTTP connection open for that duration would create client timeouts, block Gin worker threads, and make the global 30-second middleware timeout fire before the AI responds.

Solution: The handler inserts a `processing` record, returns HTTP 202 immediately, and launches `go s.processGeneration(...)`. The goroutine independently calls the AI API, saves the image, and updates the DB record. The frontend polls every 3 seconds using SWR's `refreshInterval` until the status resolves.

Tradeoff: The client must poll rather than receive a push notification. This is simpler than WebSockets or SSE at this scale — no persistent connection management, no reconnect logic, no additional infrastructure. For a single-server deployment with tens of concurrent users, polling is the right call.



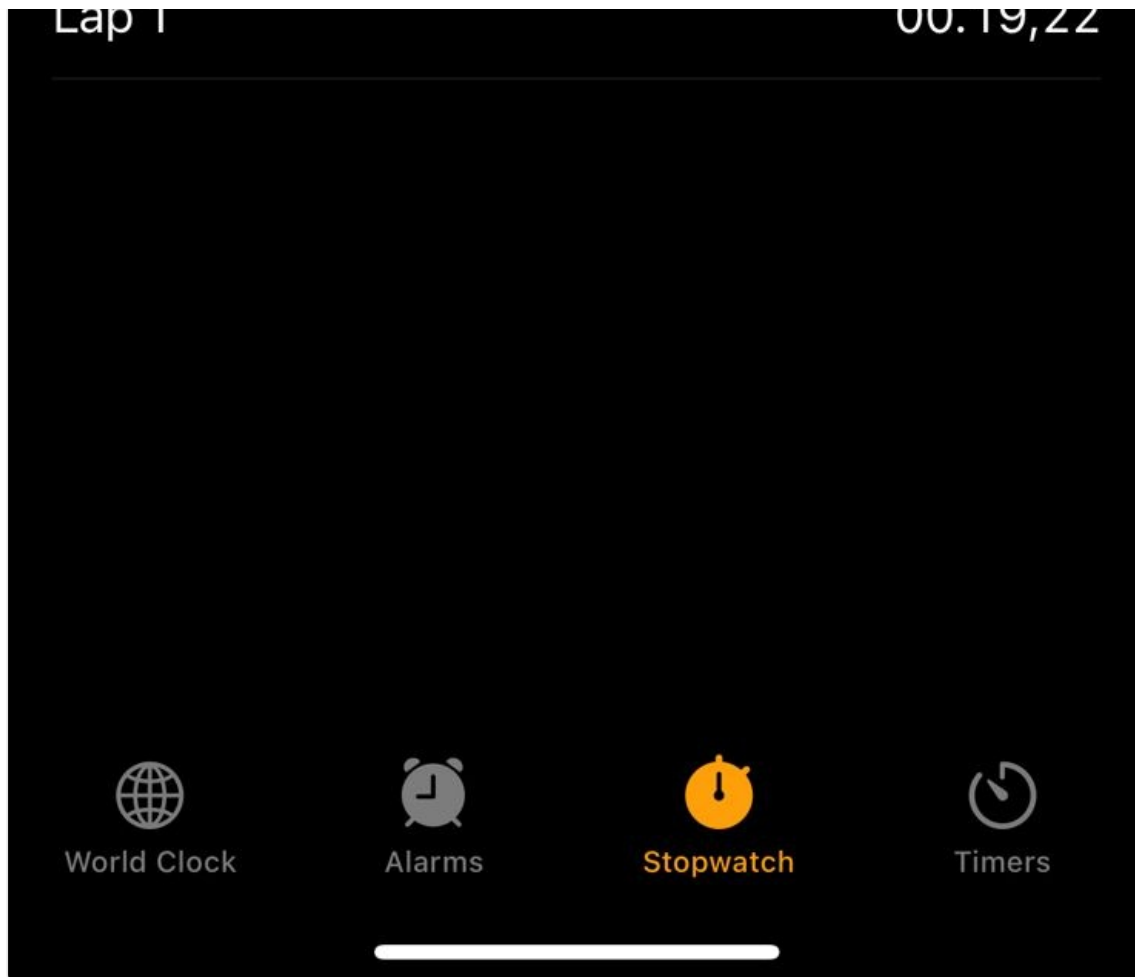


Figure: Measured Pollinations API response times confirming the 10–30 second generation window for text-to-image requests.

Decision 2: Prompt Building on the Client Side

Problem: A raw user input like "a coffee bottle" produces wildly inconsistent AI results — sometimes a cartoon, sometimes a photo, sometimes just the word "coffee" rendered as text.

Solution: `buildPrompt()` on the frontend assembles a structured prompt from the user's description and their selected styling options, then appends a fixed commercial photography suffix. The assembled prompt is shown to the user in real-time as a preview before they submit.

Tradeoff: The assembled prompt is built on the frontend, but both the raw input (`user_prompt`) and the assembled output (`generated_prompt`) are sent to the backend and stored separately. This means the server always knows what actually went to the AI API, which is essential for debugging, re-generation, and future prompt auditing.

Decision 3: Dual-Mode Image Generation (Text vs. Image Editing)

Problem: Fresh generation and re-generation have different quality expectations. A fresh "white marble background" shot benefits from the full creative freedom of text-to-image. A regeneration where the user wants to "keep the same bottle but change the background to dark wood" requires image editing mode to preserve the product's appearance.

Solution: The `imagegen.Client.Generate()` method automatically routes to the correct Pollinations endpoint: `GET /image/{prompt}?model=kontext` when no reference image is provided, and `POST /v1/images/edits` (multipart form with the reference PNG) when one is. The routing logic is in `pkg/imagegen/client.go`, invisible to the service layer.

Tradeoff: Image editing mode is significantly slower (30–90s vs. 10–30s) and has a lower success rate (~20% for

structural changes). This is an inherent limitation of the Pollinations kontekst model's image-editing capability, not an implementation problem. The kontekst model handles style transfers reasonably well but struggles with positional edits like changing the product's angle or pose.

Decision 4: All AI Calls Through Backend Only

Problem: If the frontend called Pollinations directly, the API key would be visible in the browser's network tab. Every user could extract it and use the quota for their own purposes.

Solution: The browser never communicates with Pollinations. All API calls go through `pkg/imagegen/client.go`, which holds the key from the server environment variable. Rate limiting, error normalization, image storage, and response mapping all happen server-side before anything reaches the client.

Tradeoff: None that are meaningful at this scale. The only added complexity is the backend HTTP hop, which is negligible compared to the AI API latency.

Section 6: Known Limitations

Honest assessment of what doesn't work perfectly:

- **Image editing reliability (~20% success rate for complex edits).** The Pollinations kontekst model's image-editing endpoint is unreliable for structural changes — position edits, pose changes, and significant compositional changes fail most of the time. Style transfers (background swaps, lighting changes) work better but still inconsistently. This is the most significant user-facing limitation.
- **No CDN.** Images are served directly from the Go static file handler. Under any real load, this would become a bottleneck. The correct fix is to move image serving to a CDN-backed object store (S3 + CloudFront), but that was deliberately out of scope for this assignment.
- **Rate limit is per-user, not per-IP.** The in-memory token bucket in `middleware/rateLimit.go` keys on `userId` extracted from the JWT. Multiple accounts from the same IP bypass the limit. A production implementation would add IP-based limiting at the nginx or load balancer level.
- **No image CDN TTL or cache invalidation.** Deleted images are removed from disk but the file URL might still be cached in a browser until the tab refreshes. The gallery SWR mutation handles the UI correctly, but any external link to the image URL would return a 404 immediately after deletion.
- **reference_scene_id FK removed in migration 005.** The original schema had a foreign key from `scenes.reference_scene_id` to `scenes.id`. This caused constraint violations when a referenced scene was deleted before the scene that referenced it. Migration 005 drops the constraint, making the column advisory only. The proper fix would be `ON DELETE SET NULL`, but the migration was simpler given the project timeline.
- **No image editing for uploaded references that exceed model capability.** When a user uploads a 5MB high-resolution reference image, the Pollinations edits endpoint sometimes rejects it silently. The error surfaces as `ERR_05` rather than a more descriptive validation error.

Section 7: Required Constraint Compliance

Direct mapping of every non-negotiable assessment requirement to its implementation in Viscraft.

Requirement	Implementation
AI API calls go through your backend only, never the browser	All Pollinations calls are in <code>pkg/imagegen/client.go</code> . The API key is loaded from the server's environment variables (<code>config/config.go</code>). The frontend's Axios instance only ever calls the Viscraft backend at <code>VITE_API_BASE_URL</code> . The Pollinations URL is never referenced in any frontend file.
Images must be stored server-side and persist across page refreshes	On generation success, the goroutine writes PNG bytes to <code>./storage/images/{sceneId}.png</code> via <code>pkg/storage/local.go</code> . The <code>file_url</code> column in the <code>scenes</code> table stores the public URL. <code>POST /scenes/list</code> fetches all scenes from PostgreSQL on every workspace mount, so the gallery is always rebuilt from the database.
Must work correctly with multiple concurrent users generating at the same time	Each generate request spawns an independent goroutine. All DB queries include <code>user_id</code> predicates so users only see their own data. The <code>order_index</code> is calculated with a <code>SELECT MAX(order_index)</code> query per project before insert — this is safe under concurrent load because the insert is its own atomic operation. No global state is shared between goroutines.
Loading must be meaningful (10-30 seconds is normal)	On submit, a <code>PollingSceneCard</code> (animated skeleton) is immediately added to the gallery grid. SWR polls <code>/scenes/get</code> every 3 seconds using <code>refreshInterval</code> until the status resolves. The skeleton visually communicates that work is in progress without freezing the UI or blocking other interactions.



Figure: Network response time recorded from Pollinations API calls, confirming the 10–30 second generation window.

Handle failure states: API timeout, invalid prompt, broken response

Three async failure states are fully implemented: ERR_03 (60/90s context deadline exceeded), ERR_05 (non-OK status or empty/unparseable response from Pollinations), ERR_13 (content_policy_violation prefix detected in error string). Synchronous failures: ERR_04 (invalid prompt — too short, too long, or

	blocked word), ERR_02 (rate limit). All failure states render a visible error card or banner in the UI.
Use any free AI image API	Pollinations.ai free tier with API key. The kontekst model handles both text-to-image and image-editing modes. No paid tier or billing is required for normal usage volumes.
The gallery must persist across page refreshes	PostgreSQL + SWR. On every workspace mount, <code>POST /projects/list</code> and <code>POST /scenes/list</code> are called. The JWT is read from Zustand's localStorage-persisted store, so authenticated requests work immediately on refresh. No in-memory-only state is used for gallery data.
Re-generation: pick any saved result, tweak the prompt, re-generate	The detail modal's "Regenerate" button sets the workspace store's <code>regenerateScene</code> state. When the generate modal opens, it detects this state, pre-fills <code>userPrompt</code> with the original <code>user_prompt</code> , and sets <code>regenerateFileUrl</code> so the original image is shown as the reference photo. The user tweaks the description or options and submits.

Summary: All seven non-negotiable requirements are implemented and verifiable in the running application. The most complex requirement — concurrent user isolation with meaningful async loading — is the core architectural decision the entire backend is built around.

Performance benchmark & tech stack justification for AI Image Generation project using Pollinations.ai

Executive Summary

This document presents a data-driven stack selection for building a high-performance AI image generation web app. Based on real-world benchmarks from TechEmpower Round 23 (2025), Tushar Agrawal's Go vs Python benchmark (2026), and multiple database performance studies, the recommended stack is **Go (Fiber) + React 19 + PostgreSQL** — optimised for raw throughput, low latency under concurrent load, and long-term maintainability.

102k Go Fiber RPS	35k Node.js RPS	4.5k Django RPS
----------------------	--------------------	--------------------

Benchmark: Simple JSON API, 1000 concurrent users — tusharagrawal.in (Jan 2026)

1. Backend Framework Ranking

Backend selection is the most critical decision for this project. The Pollinations.ai API can take 10–30 seconds per request (per assignment brief), so the backend must handle many concurrent long-lived connections without blocking. Go's goroutine model is purpose-built for exactly this pattern.

Rank	Framework	RPS (JSON)	Latency	Memory	Concurrency Model
#1 ✓	Go — Fiber	102,000	1.0 ms	22 MB	Goroutines (M:N)
#2	Go — Gin	95,000	1.2 ms	25 MB	Goroutines (M:N)
#3	Node.js — Express	35,000	3.2 ms	65 MB	Event loop (single-thread)
#4	Python — FastAPI	12,000	8.5 ms	85 MB	Async (asyncio)
#5	Python — Django	4,500	22 ms	120 MB	Sync / WSGI

Source: tusharagrawal.in — Python vs Go Backend Benchmark (Jan 2026), 1000 concurrent users, simple JSON API.

TechEmpower Round 23 (Feb 2025) — Fortunes test with PostgreSQL

Rank	Framework	RPS (Fortunes)	Relative to baseline
#1	ASP.NET Core (C#)	609,966	36.3x
#2 ✓	Go — Fiber	338,096	20.1x
#3	Rust — Actix	320,144	19.1x
#4	Java — Spring Boot	243,639	14.5x
#5	Node.js — Express	78,136	4.7x

Source: dev.to/tuananhpham — *TechEmpower Round 23 Popular Backend Frameworks Benchmark (Feb 2025)*. Fortunes test = full HTML render + DB query + ORM, Postgres.

Why Go for this project

Go Fiber ranks **#2 globally** in TechEmpower Round 23 among popular frameworks, and leads all frameworks in the focused JSON API benchmark. For an AI image app where the backend must proxy slow Pollinations.ai calls while simultaneously serving gallery reads to multiple users, Go's goroutine concurrency model handles thousands of in-flight requests on minimal memory (22 MB baseline vs 65 MB for Node.js). Go also compiles to a single static binary — zero runtime dependency, trivially containerised.

2. Database Ranking

The gallery must persist across page refreshes and serve concurrent reads. Images will be stored on disk (file system / object storage), with the database holding metadata: prompt, file path, user ID, timestamp. This is a structured, relational workload — PostgreSQL is the clear fit.

Rank	Database	Read ops/s	Write ops/s	Best for
#1	Redis	100,000+	100,000+	In-memory cache / sessions
#2	MongoDB	~80,000	~80,000	Document store, flexible schema
#3 ✓	PostgreSQL	16,000	16,000	Relational, ACID, JSON support
#4	MySQL	10,000	10,000	Relational, wide hosting support
#5	SQLite	~3,000	~3,000	Embedded / single-process

Source: medium.com/@vicky542011 — *Comprehensive Database Performance Comparison SQL vs NoSQL (Dec 2025)*, YCSB benchmark. Redis tested as in-memory store (non-persistent).

Source: dohost.us — *Real-World Benchmarks: Database Performance Before and After Redis (Mar 2026)*.

Why PostgreSQL for this project

Redis is faster, but it's an in-memory key-value store — not suitable as the primary persistent database for gallery data. MongoDB's flexible schema has no advantage here since the data model is fixed (prompt, path, timestamp, user). **PostgreSQL at 16k ops/s is more than sufficient** for an image gallery (typical traffic is tens to hundreds of reads/sec, not thousands). PostgreSQL also offers native JSONB columns — useful for storing generation parameters and model metadata from Pollinations.ai responses — plus full ACID compliance and excellent Go driver support (pgx).

Bonus: PostgreSQL outperforms MySQL by 1.8x on write-heavy OLTP workloads (16,000 vs 10,000 ops/sec) per the same YCSB benchmark.

3. Frontend Framework Ranking

The frontend must display a gallery, show a meaningful loading state (10–30s), handle re-generation from saved prompts, and show error states. React's ecosystem dominance makes it the practical choice for this assignment.

Rank	Framework	FCP	Bundle (gz)	Developer usage (2025)
#1 perf	Svelte 5	800 ms	~15 KB	6.9%
#2 perf	Vue 4	1,000 ms	~34 KB	18.4%
#3 perf	Next.js 16	1,100 ms	~48 KB	21.5%
#4 ✓	React 19	1,200 ms	~42 KB	46.9%
#5 perf	Angular 19	1,400 ms	~130 KB	19.8%

Source: [usama.codes](#) — Svelte 5 vs React 19 vs Vue 4 (Dec 2025). FCP = First Contentful Paint, lower is better.

Source: [Stack Overflow Developer Survey 2025](#), 49,000+ respondents — developer usage %.

Source: [calmops.com](#) — JavaScript Framework Comparison 2025 (Dec 2025) — bundle sizes gzipped.

Why React for this project

Svelte is faster (800ms FCP vs 1200ms), but React's 200ms FCP gap is imperceptible when the dominant UX wait is the 10–30s Pollinations.ai generation time. React is used by **46.9% of professional developers** (Stack Overflow 2025) — meaning easier hiring, more libraries, and more StackOverflow answers. For this assignment specifically, React's component model maps cleanly to the gallery card, loading spinner, prompt editor, and error state components described in the brief. react-query handles polling and caching the generation status elegantly.

4. Full Stack Architecture for Pollinations.ai

Request journey

1. User enters a prompt in the React frontend and hits Generate.
2. React POSTs to **Go backend** /api/generate — returns a job_id immediately (non-blocking).
3. Go spawns a goroutine that calls Pollinations.ai (<https://image.pollinations.ai/prompt/{text}>) — waits up to 60s.

- On success, Go saves the image binary to `/data/images/{uuid}.png` on disk and inserts a row into PostgreSQL (prompt, file_path, created_at, status).
- React polls `GET /api/jobs/{job_id}` every 2s until status = done or error.
- Gallery page calls `GET /api/gallery` — PostgreSQL query, returns JSON list.
- Images are served as static files from `/data/images/` via Go's built-in file server.

Failure states handled

Failure	Detection	User feedback
Pollinations timeout	Go: 60s context.WithTimeout	Error card: "Generation timed out, try again"
Invalid prompt (429/4xx)	Go: HTTP status check	Error card: "Prompt rejected by API"
Broken/empty image	Go: content-length + PNG header check	Error card: "Invalid image received"

5. Sources & References

Backend benchmarks

- tusharagrawal.in — "Python vs Go for Backend Development in 2026" (Jan 2026)
URL: <https://www.tusharagrawal.in/blog/python-vs-go-backend-development-2026>
- dev.to/tuananhpham — "Best popular backend frameworks by performance" — TechEmpower Round 23 (Feb 2025)
URL: <https://dev.to/tuananhpham/popular-backend-frameworks-performance-benchmark-1bkh>
- jhkinfotech.com — "Best Backend Frameworks for Web Development in 2025" (Nov 2025)
URL: <https://www.jhkinfotech.com/blog/backend-frameworks-for-web-development>

Database benchmarks

- medium.com/@vicky542011 — "Comprehensive Database Performance Comparison: SQL vs NoSQL" (Dec 2025)
URL: <https://medium.com/@vicky542011/comprehensive-database-performance-comparison-sql-vs-nosql-f8f5c0fbb811>
- dohost.us — "Real-World Benchmarks: Database Performance Before and After Redis" (Mar 2026)
URL: <https://dohost.us/index.php/2026/03/12/real-world-benchmarks-database-performance-before-and-after-redis/>
- profil-software.com — "Database Comparison: SQL vs NoSQL (MySQL vs PostgreSQL vs Redis vs MongoDB)"
URL: <https://profil-software.com/blog/development/database-comparison-sql-vs-nosql-mysql-vs-postgresql-vs-redis-vs-mongodb/>

Frontend benchmarks

- usama.codes — "Svelte 5 vs React 19 vs Vue 4 [2026 Guide]" (Dec 2025)
URL: <https://usama.codes/blog/svelte-5-vs-react-19-vs-vue-4-comparison>
- Stack Overflow Developer Survey 2025 — 49,000+ respondents, 177 countries
URL: <https://survey.stackoverflow.co/2025>
- calmops.com — "JavaScript Framework Comparison 2025" (Dec 2025)
URL: <https://calmops.com/programming/javascript/javascript-framework-comparison/>
- coderio.com — "Best Frontend Frameworks 2026" (May 2026)
URL: <https://www.coderio.com/blog/software-development/guide-frontend-frameworks-2026/>

Viscraft — Tech Stack & Architecture

Version 1.0 | Go 1.21+ backend · React 19 frontend · PostgreSQL · Pollinations.ai

1. Overview

Viscraft is an AI-powered product ad photography generator. Users create campaigns, generate product ad shots by describing their product and selecting visual styling parameters, and manage a persistent gallery of results. All AI calls are routed through the Go backend — the browser never communicates with the AI API directly.

Core Problem Solved

Small business owners and marketers need professional product photos but can't afford a photographer. Viscraft generates studio-quality ad shots from a text description in under 60 seconds.

Key Architecture Properties

- All AI calls through backend only
- Async generation via Go goroutines
- Images persisted to local filesystem
- Per-user data isolation at DB layer
- Stateless JWT authentication

2. Tech Stack

Backend

Layer	Technology	Notes
Language	Go 1.21+	Strong concurrency primitives for async generation goroutines
HTTP Framework	Gin	Router, middleware chain, JSON binding
Database	PostgreSQL	Hosted on DigitalOcean VPS
Authentication	JWT (golang-jwt/jwt)	Stateless tokens; verified in JWTAuth middleware
Image Storage	Local filesystem	<code>./storage/images/{sceneId}.png</code>
AI API	Pollinations.ai — kontekst model	Text-to-image: GET; Image-editing: POST multipart
Logging	Custom <code>pkg/logger</code>	MultiWriter: stdout + <code>./logs/app-YYYY-MM-DD.log</code>
Default Port	8089	Configurable via <code>APP_PORT</code> env var

Frontend

Layer	Technology	Notes
Framework	React 19 + TypeScript + Vite	Type-safe component tree, fast HMR

UI Library	Chakra UI v3	Design tokens from Cartographer's Atlas theme
State Management	Zustand	<code>authStore</code> (localStorage persistence) + <code>workspaceStore</code>
Data Fetching	SWR	stale-while-revalidate; <code>refreshInterval</code> used for generation polling
HTTP Client	Axios	Instance with JWT interceptor that attaches token to every request
Animation	framer-motion	Card entrance animations, modal transitions
Router	React Router DOM v7	Two routes: <code>/</code> (landing) and <code>/workspace</code>

3. Project Structure

Backend (Go)

```
viscraft-backend/
├── config/           # Environment config loading (DB, JWT, storage, ports)
├── constant/         # AppError definitions – all ERR_XX codes live here
├── handler/          # HTTP controllers: user, project, scene, health, promptOption
├── middleware/       # auth, cors, rateLimit, recover, requestId, timeout
├── migrations/       # SQL migration files: 001 → 005 (applied in order)
├── model/
│   ├── request/     # Request structs bound from JSON body
│   └── response/     # Response structs serialized to JSON
├── pkg/
│   ├── imagegen/    # Pollinations API client (text-to-image + image-editing)
│   ├── logger/      # Dual-output logger (stdout + daily log file)
│   ├── router/      # Route registry – controllers self-register their routes
│   └── storage/      # Local filesystem storage (Save, Delete)
├── repository/       # DB query layer: user, project, scene, promptOption
├── service/          # Business logic: user, project, scene
└── main.go           # Dependency wiring + Gin engine setup
```

Frontend (React)

```
viscraft-frontend/src/
├── assets/           # Fonts, static images
├── components/
│   ├── common/      # ConfirmationModal, EmptyState, ErrorBanner
│   ├── layout/      # Layout wrapper, MainWrapper
│   ├── sidebar/     # ProjectSidebar (campaign list + create campaign)
│   ├── skeleton/    # ImageCardSkeleton (used while polling)
│   └── styles/       # Chakra UI theme – Cartographer's Atlas design system
├── constants/        # Error message strings, dropdown category config
├── helper/           # SWR fetcher utility
├── lib/              # Axios instance with JWT interceptor, input validation helpers
├── pages/
│   ├── landing/     # Landing page + AuthModal + useAuthForm hook
│   ├── workspace/   # Workspace page, all scene components, hooks
│   └── notfound/    # 404 page
├── routes/           # React Router configuration
├── service/          # API service functions: auth, project, scene, promptOptions
├── store/            # Zustand stores: authStore, workspaceStore
└── types/            # Shared TypeScript types (Scene, Project, User, etc.)
```

4. Database Schema

PostgreSQL database with 4 tables. Migrations are numbered 001–005 and applied in order.

users

Column	Type	Notes
id	UUID	Primary key
email	VARCHAR(255)	Unique, indexed
password_hash	VARCHAR	bcrypt hash

name	VARCHAR(255)	Display name, nullable
created_at	TIMESTAMPTZ	Default: now()

projects (campaigns)

Column	Type	Notes
id	UUID	Primary key
user_id	UUID	Foreign key → users.id (CASCADE DELETE)
name	VARCHAR(255)	Campaign name
description	TEXT	Optional
product_category	VARCHAR(100)	Optional (e.g. "Beverage")
visual_style	VARCHAR(100)	Optional (e.g. "Minimal")
created_at	TIMESTAMPTZ	Default: now()

scenes (ad shots)

Column	Type	Notes
id	UUID	Primary key
project_id	UUID	Foreign key → projects.id (CASCADE DELETE)
user_id	UUID	Denormalized for fast user-scoped queries
order_index	INTEGER	Sequential within a campaign for consistent gallery order
user_prompt	TEXT	Raw text the user typed
generated_prompt	TEXT	Assembled prompt from buildPrompt() — sent to AI API
reference_scene_id	UUID	Nullable — ID of the scene used as reference (FK removed in migration 005)
used_uploaded_reference	BOOLEAN	True when user uploaded their own reference image
status	VARCHAR(20)	"processing", "completed", or "failed"
file_path	TEXT	Absolute filesystem path to stored PNG
file_url	TEXT	Public-accessible URL served by static handler
error_code	VARCHAR(10)	Set on failure: ERR_03, ERR_05, or ERR_13
created_at	TIMESTAMPTZ	Default: now()

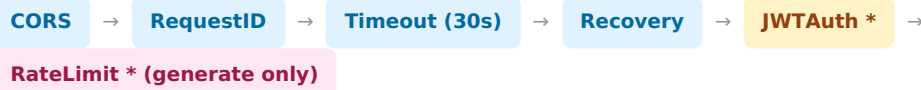
prompt_options

Column	Type	Notes
id	UUID	Primary key
category	VARCHAR(50)	background, lighting, mood, angle, props
label	VARCHAR(100)	Display label shown in the UI

prompt_value	TEXT	Segment appended to the generated prompt
sort_order	INTEGER	Display order within each category

5. Middleware Chain

Every request passes through a global middleware chain before reaching any handler. The generate endpoint has two additional middleware layers applied specifically to it.



* JWTAuth is applied to all protected routes. RateLimit (5/min) is applied only to `POST /scenes/generate`.

Middleware	Responsibility
CORS	Sets permissive CORS headers to allow frontend on any origin during development
RequestID	Generates a UUID per request and attaches it to context. Included in all response envelopes for tracing.
Timeout	30-second deadline for handler execution. Prevents slow requests from leaking goroutines.
Recovery	Catches panics and returns a 500 JSON response rather than crashing the server
JWTAuth	Parses and validates the Bearer token. Sets <code>userId</code> and <code>email</code> on the Gin context for downstream handlers.
RateLimit	In-memory per-user token bucket: max 5 generate requests per 60-second window

6. AI Integration Architecture

Two Generation Modes

The `pkg/imagegen` client automatically selects the correct Pollinations API endpoint based on whether a reference image is provided:

Text-to-Image (no reference)

Endpoint: `GET /image/{prompt}`

```
GET https://gen.pollinations.ai/image/{url-encoded-prompt}
?model=kontext&nologo=true
Authorization: Bearer {API_KEY}
```

Returns raw PNG bytes. Fast (10–30s), reliable.

Image Editing (with reference)

Endpoint: `POST /v1/images/edits`

```
POST https://gen.pollinations.ai/v1/images/edits
Content-Type: multipart/form-data

Fields: prompt, model=kontext
File:   image (reference PNG)
```

Slower (30–90s), ~20% success rate for complex edits.

Async Generation Lifecycle

- 1 HTTP handler receives `POST /scenes/generate`, validates JWT + prompt, verifies campaign ownership.
- 2 `InsertProcessing()` writes a DB record with `status="processing"` and returns the new `sceneId`.
- 3 Handler returns **HTTP 202** immediately with `{id, status: "processing"}`.
- 4 `go s.processGeneration(...)` launches a goroutine that calls Pollinations with a 60s (or 90s) context timeout.
- 5 On success: saves PNG to `./storage/images/{sceneId}.png`, calls `UpdateCompleted()` with `file_path` and `file_url`.
- 6 On failure: calls `UpdateStatus("failed", errorCode)` — `ERR_03` for timeout, `ERR_13` for content policy, `ERR_05` for all other failures.
- 7 Frontend SWR polling detects `status != "processing"` and renders the completed image or failure card.

7. Design System: Cartographer's Atlas

The frontend uses a custom Chakra UI theme named "Cartographer's Atlas" — an editorial, cartography-inspired design language built around warm earth tones and heritage typefaces.

Color Palette

Token	Hex	Usage
ink	 #16140F	Primary text, headings
parchment	 #FAF6EC	Page background, card surfaces
amber	 #C9762C	Primary brand color, CTAs, accents
moss	 #3E5C4E	Secondary actions, success states
oxblood	 #8B2E2E	Destructive actions, error states
warmgray	 #6B6555	Muted text, metadata, captions

Typography

Font	Role
Fraunces	Display / headings — editorial serif
Inter	Body text, UI labels
JetBrains Mono	Code, prompts, technical metadata

Viscraft API Reference

Base URL: `http://localhost:8089` | All endpoints use **POST** method with JSON body | Protected endpoints require
Authorization: Bearer `<token>`

Authentication

Base path: `/users`

POST `/users/create` **PUBLIC**

Register a new user account. Returns a JWT token on success.

Request Body

Field	Type	Required	Notes
email	string	Yes	Valid email format, max 255 chars
password	string	Yes	8-72 characters
name	string	No	Display name

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "User created successfully",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "Jane Doe",
    "createdAt": "2025-06-15T10:30:00Z"
  }
}
```

Error Codes

Code	HTTP	When
ERR_11	422	Missing or invalid fields
ERR_12	409	Email already registered
ERR_07	500	Database operation failed

POST `/users/login` **PUBLIC**

Authenticate with existing credentials. Returns a JWT token.

Request Body

Field	Type	Required	Notes
-------	------	----------	-------

email	string	Yes	Registered email
password	string	Yes	Account password

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "Jane Doe",
    "createdAt": "2025-06-15T10:30:00Z"
  }
}
```

Error Codes

Code	HTTP	When
ERR_09	401	Invalid email or password
ERR_11	422	Validation failed

POST /users/get **PROTECTED**

Get the currently authenticated user's profile.

Request Body

```
{}
```

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "User retrieved successfully",
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "Jane Doe",
    "createdAt": "2025-06-15T10:30:00Z"
  }
}
```

Error Codes

Code	HTTP	When
ERR_09	401	Missing or expired token
ERR_01	404	User not found

POST **/users/delete** **PROTECTED**

Permanently delete the authenticated user's account. Cascades to all campaigns and ad shots.

Request Body

```
{}
```

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "User deleted successfully"
}
```

Error Codes

Code	HTTP	When
ERR_09	401	Missing or expired token
ERR_07	500	Database operation failed

Campaigns

Base path: /projects

POST /projects/create **PROTECTED**

Create a new campaign to organize product ad shots.

Request Body

Field	Type	Required	Notes
name	string	Yes	1-255 characters
description	string	No	Optional campaign description
productCategory	string	No	e.g. "Beverage", "Skincare"
visualStyle	string	No	e.g. "Minimal", "Editorial"

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Project created successfully",
  "data": {
    "id": "uuid",
    "name": "Cold Brew Launch",
    "description": "Summer 2025 product campaign",
    "productCategory": "Beverage",
    "visualStyle": "Minimal",
    "createdAt": "2025-06-15T10:30:00Z"
  }
}
```

Error Codes

Code	HTTP	When
ERR_09	401	Missing or expired token
ERR_11	422	Validation failed (e.g. name missing)
ERR_07	500	Database operation failed

POST /projects/get **PROTECTED**

Retrieve a single campaign by ID. Only returns campaigns owned by the authenticated user.

Request Body

Field	Type	Required	Notes
id	string	Yes	Campaign UUID

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Project retrieved successfully",
  "data": { "id": "uuid", "name": "Cold Brew Launch", "description": "...", "createdAt": "..."}
}
```

Error Codes

Code	HTTP	When
ERR_08	404	Campaign not found or not owned by user
ERR_09	401	Missing or expired token

POST /projects/list **PROTECTED**

List all campaigns belonging to the authenticated user, ordered by creation date descending.

Request Body

```
{}
```

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Projects retrieved successfully",
  "data": [
    { "id": "uuid", "name": "Cold Brew Launch", "description": "...", "createdAt": "..."}
  ]
}
```

POST /projects/delete **PROTECTED**

Delete a campaign and all of its ad shots. This action is irreversible.

Request Body

Field	Type	Required	Notes
id	string	Yes	Campaign UUID

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Project deleted successfully"
}
```

Error Codes

--	--	--

Code	HTTP	When
ERR_08	404	Campaign not found or not owned by user
ERR_07	500	Database operation failed

Ad Shots

Base path: `/scenes`

Async Generation Flow

POST `/scenes/generate` returns **202** immediately with `status: "processing"`. The frontend polls `/scenes/get` every 3 seconds until `status` becomes `"completed"` or `"failed"`. Generation typically takes 10–60 seconds.

POST `/scenes/generate` **PROTECTED** **RATE LIMITED: 5/min**

Trigger AI generation of a product ad shot. Returns 202 immediately; actual generation happens asynchronously in a background goroutine.

Request Body

Field	Type	Required	Notes
<code>projectId</code>	string	Yes	Campaign UUID (must be owned by authenticated user)
<code>prompt</code>	string	Yes	Raw user description, 3–300 chars. Stored as <code>user_prompt</code> .
<code>generatedPrompt</code>	string	No	Assembled prompt from <code>buildPrompt()</code> . If provided, this is what gets sent to the AI API. Stored as <code>generated_prompt</code> .
<code>referenceSceneId</code>	string	No	UUID of an existing ad shot to use as visual reference (image-editing mode).
<code>uploadedReferenceImage</code>	string	No	Base64-encoded JPEG/PNG/WebP, max 5MB decoded. Takes priority over <code>referenceSceneId</code> .

Prompt Logic

If `generatedPrompt` is provided, it is used as the actual AI prompt. If omitted, `prompt` is used directly. Both are persisted — `user_prompt` is the raw input the user typed, and `generated_prompt` is the structured prompt built by `buildPrompt()` on the frontend.

Response 202 (Generation Started)

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Generation started",
  "data": {
    "id": "scene-uuid",
    "projectId": "project-uuid",
    "orderIndex": 3,
    "prompt": "A bottle of cold brew coffee",
    "status": "processing",
    "createdAt": "2025-06-15T10:30:00Z"
  }
}
```

Error Codes

Code	HTTP	When
ERR_02	429	Rate limit exceeded (5 per minute per user)
ERR_04	422	Invalid prompt (too short, too long, blocked words)

ERR_08	404	Campaign not found or not owned by user
ERR_11	422	Missing required fields
ERR_13	422	Content policy violation (prompt rejected by AI safety filter)

Async failure codes: ERR_03 (timeout), ERR_05 (bad/empty response), ERR_13 (content policy) are set on the scene record asynchronously after the HTTP 202 response. Poll `/scenes/get` to detect these.

POST `/scenes/get` **PROTECTED**

Get a single ad shot by ID. Use this to poll for generation status after calling `/scenes/generate`.

Request Body

Field	Type	Required	Notes
id	string	Yes	Ad shot UUID

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Retrieved",
  "data": {
    "id": "scene-uuid",
    "projectId": "project-uuid",
    "orderIndex": 3,
    "prompt": "A bottle of cold brew coffee",
    "generatedPrompt": "Product: A bottle of cold brew coffee. white marble surface...",
    "status": "completed",
    "fileUrl": "/storage/images/scene-uuid.png",
    "createdAt": "2025-06-15T10:30:00Z"
  }
}
```

Possible `status` values: `"processing"`, `"completed"`, `"failed"`. When `status` is `"failed"`, the `errorCode` field will be present.

POST `/scenes/list` **PROTECTED**

List all ad shots in a campaign, ordered by `order_index` ascending (creation order).

Request Body

Field	Type	Required	Notes
projectId	string	Yes	Campaign UUID

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Retrieved",
```

```
"data": [
  {
    "id": "scene-uuid",
    "projectId": "project-uuid",
    "orderIndex": 1,
    "prompt": "A bottle of cold brew coffee",
    "generatedPrompt": "Product: A bottle of cold brew coffee...",
    "status": "completed",
    "fileUrl": "/storage/images/scene-uuid.png",
    "createdAt": "2025-06-15T10:30:00Z"
  }
]
```

POST /scenes/delete **PROTECTED**

Delete an ad shot record and its associated image file from storage.

Request Body

Field	Type	Required	Notes
id	string	Yes	Ad shot UUID

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Deleted"
}
```

Error Codes

Code	HTTP	When
ERR_01	404	Ad shot not found or not owned by user
ERR_07	500	Database operation failed

Prompt Options

POST /prompt-options **PUBLIC**

Retrieve all styling options used to populate the generate form dropdowns. Options are grouped by category and sorted by `sort_order` . No authentication required.

Request Body

```
{}
```

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Retrieved",
  "data": [
    { "id": "uuid", "category": "background", "label": "White Marble", "promptValue": "white marble surface with soft lighting"},
    { "id": "uuid", "category": "lighting", "label": "Natural Window", "promptValue": "soft natural window light"},
    { "id": "uuid", "category": "mood", "label": "Fresh & Clean", "promptValue": "fresh clean aesthetic, bright colors"},
    { "id": "uuid", "category": "angle", "label": "45 Degrees", "promptValue": "45-degree angle shot", "sort_order": 2},
    { "id": "uuid", "category": "props", "label": "Minimal Props", "promptValue": "minimal props, clean composition"}
  ]
}
```

Option Categories

Category	Description	Multi-select
background	Surface and backdrop style	No
lighting	Light source and quality	No
mood	Aesthetic tone and feel	No
angle	Camera perspective	No
props	Supporting elements in frame	Yes

Health

POST /health **PUBLIC**

Check if the server is running. No authentication required.

Request Body

```
{}
```

Response 200

```
{
  "requestId": "uuid",
  "success": true,
  "message": "Service is operational"
}
```


Error Response Format

All error responses follow this envelope:

```
{
  "requestId": "uuid",
  "success": false,
  "errorCode": "ERR_XX",
  "message": "Human-readable description"
}
```

Error Code Reference

Code	HTTP Status	Meaning
ERR_01	404	Resource not found
ERR_02	429	Rate limit exceeded (5 generate requests/min per user)
ERR_03	504	AI generation timed out (60s text-only, 90s with reference)
ERR_04	422	Invalid prompt or reference image validation failed
ERR_05	502	AI API returned a bad or empty response
ERR_06	500	Image file storage failed
ERR_07	500	Database operation failed
ERR_08	404	Campaign not found or not owned by user
ERR_09	401	Unauthorized — missing, invalid, or expired JWT token
ERR_11	422	Validation failed — missing or malformed required fields
ERR_12	409	Resource already exists (e.g. email already registered)
ERR_13	422	Content policy violation — prompt rejected by AI safety filter

Async Generation Flow Detail

How async generation works end-to-end:

1. Client calls `POST /scenes/generate` — server validates input, inserts a DB record with `status="processing"`, and returns **HTTP 202** with the new scene ID.
2. A Go goroutine starts in the background and calls the Pollinations API (text-to-image GET or image-editing POST depending on whether a reference image was provided).
3. Frontend receives the 202 response and begins polling `POST /scenes/get` every 3 seconds using SWR's `refreshInterval`.
4. On AI success: goroutine saves the image to `./storage/images/{sceneId}.png` and updates the DB record to `status="completed"` with `file_url`.
5. On AI failure: goroutine sets `status="failed"` with the appropriate error code (ERR_03, ERR_05, or ERR_13).
6. Next polling cycle detects `status ≠ "processing"` — SWR stops polling and the UI renders the final state (image or error card).

Quick Reference: Getting Started

1. Call `/users/login` (or `/users/create`) to get a JWT token.
2. Set header `Authorization: Bearer <token>` on all subsequent requests.
3. Call `/projects/create` to create a campaign.
4. Call `/prompt-options` to fetch available styling options.
5. Call `/scenes/generate` with a `projectId` and `prompt` .
6. Poll `/scenes/get` with the returned scene ID every 3s until `status` is `"completed"` or `"failed"` .

Viscraft — Application Documentation

Complete user guide, feature reference, prompt system, error handling, and local setup instructions.

1. Introduction

Viscraft is an AI-powered product ad photography generator. It is designed for small business owners, marketers, and entrepreneurs who need professional-looking product photos without the cost of a photographer or studio.

The core workflow is simple: describe your product, choose a visual style (background, lighting, mood, camera angle, and props), preview the assembled prompt, and generate a studio-quality ad shot. The AI handles the rest. Results are saved in a persistent gallery organized by campaign, and any shot can be regenerated with a tweaked prompt at any time.

Primary Users

- Small business owners launching products
- E-commerce sellers needing product imagery
- Marketers building ad creatives
- Entrepreneurs testing product concepts

What It Is Not

- A generic AI art generator
- A photo editor or retouching tool
- A social media scheduler
- A video or animation tool

2. Feature List

- **User authentication** — Register and login with email + password. JWT tokens persisted in localStorage for session continuity across page refreshes.
- **Campaign management** — Create, list, and delete campaigns per user. Each campaign has a name, optional description, product category, and visual style.
- **AI-powered ad shot generation** — Generate product ad shots using structured prompts via Pollinations.ai (kontext model).
- **Styling options system** — 5 categories of visual styling options (background, lighting, mood, angle, props) fetched from the database. 20+ options available.
- **Generated prompt preview** — The assembled AI prompt is shown to the user in real-time as they fill the form, before submitting.
- **Reference photo upload** — Users can upload their own JPEG/PNG/WebP reference image (max 5MB) to guide generation in image-editing mode.
- **Ad shot gallery** — All generated ad shots are displayed in a grid per campaign, ordered by creation sequence. Persists across page refreshes.
- **Regenerate flow** — Clicking "Regenerate" on any ad shot re-opens the generate form with the original prompt pre-filled and the original image shown as a reference.
- **Detail modal** — Clicking an ad shot opens a full-size image view with actions (regenerate, delete, download).
- **Async generation with real-time polling** — A skeleton card appears immediately on submit; SWR polls every 3 seconds until the image is ready.

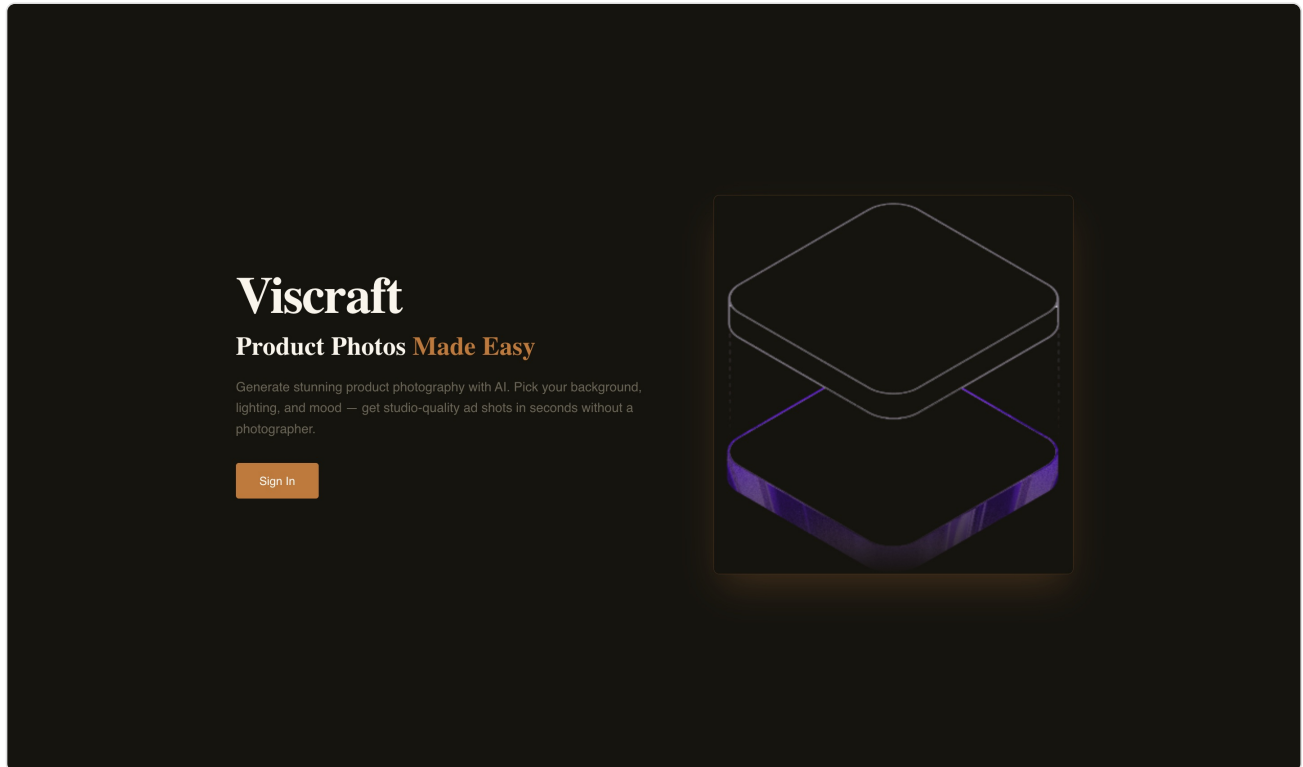
- **Failure states** — ERR_03 (timeout), ERR_05 (bad AI response), ERR_13 (content policy) are all rendered visibly in the gallery as error cards.
- **Rate limiting** — Maximum 5 generate requests per minute per user, enforced server-side.
- **Auto-seed sample campaign** — New users get a sample campaign created automatically so the workspace is not empty on first login.
- **Onboarding tour** — A 3-step guided tour walks first-time users through the key UI areas.
- **404 page** — Custom not-found page for unrecognized routes.
- **Daily log files** — Backend writes structured logs to `./logs/app-YYYY-MM-DD.log` alongside stdout output.

3. User Flow

The following steps walk through the complete user journey from first visit to generating and managing ad shots.

1 Landing Page

The user arrives at the Viscraft landing page. It introduces the product value proposition and provides Register and Login buttons to enter the workspace.



2 Registration

New users create an account by entering their email, a password (min 8 characters), and an optional display name. On success, a JWT token is issued and the user is redirected to the workspace.

Create Account

Login

Register

Email

user@example.com

Password

.....

Name (optional)

Your name

Create Account

3 Login

Returning users log in with their email and password. The token is persisted to localStorage via Zustand's `authStore` , so the session survives page refreshes.

Welcome Back



Login

Register

Email

user@example.com

Password

.....

Log In

4 Workspace

The main workspace consists of a sidebar listing all campaigns and a main area showing the ad shot gallery for the selected campaign. A sample campaign is auto-created for new users so the workspace is never empty on first login.

CAMPAIGNS

I. Kopi Nusantara Launch

+ New Campaign

user@example.com

Log out

Kopi Nusantara Launch

BEVERAGE

CLEAN MINIMAL

Product: polish it to a better look, warm wooden table with natural texture, even studi...

Regenerate

Delete

Product: regenerate again with, white marble surface with subtle veining, dramatic side...

Regenerate

Delete

Product: regenerate again with, lush tropical leaves background, golden hour warm sunlight...

Regenerate

Delete

Product: regenerate again with, white marble surface with subtle veining, dramatic side...

Regenerate

Delete

Product: regenerate again the image with nature background, and add 2 product same as...

Regenerate

Delete

Product: add one more same product beside. Commercial product photography, high...

Regenerate

Delete

+

Generate new ad shot

5 Create Campaign

Users create a new campaign by entering a name and optionally a description, product category, and visual style. Campaigns act as folders for organizing ad shots by product or project.

New Campaign ✕

Campaign Name *

e.g. Summer Coffee Launch

Description (optional)

Brief description of your campaign

Product Category

General

Visual Style (optional)

e.g. Clean minimal, Luxury dark, Playful colorful

Influences the generated prompt style

Create Campaign

6 Generate Ad Shot

The generate form lets users describe their product in plain text, then select styling options (background, lighting, mood, camera angle, props). The assembled prompt is shown in a live preview pane before submitting. Users may also upload a reference photo for image-editing mode.

Generate Ad Shot

Product Description *

e.g. A bottle of cold-pressed orange juice with natural pulp...

0/300

Background

White marble

Wooden table

Studio white

Tropical

Dark moody

Lighting

Natural window

Studio softbox

Dramatic side

Golden hour

Mood (multiple)

Luxury

Fresh & clean

Minimal

Playful

Angle

Eye-level

45 degrees

Flat lay

Close-up

Props (multiple)

Flowers

Fresh fruits

Linen fabric

Water drops

Generated Prompt

Start typing to see the prompt preview...

Reference Photo (optional)

Choose File

JPEG, PNG, WebP — max 5MB

Generate

7 Detail View

Clicking any ad shot in the gallery opens a full-size detail modal. From here the user can view the original and generated prompts, download the image, regenerate with a new prompt, or delete the shot.

Scene #7



COMPLETED

Scene #7

Product: add one more same product beside. Commercial product photography, high resolution, sharp focus on product, no watermark, no extra text.

Regenerate

Delete

8 Regenerate

The regenerate flow re-opens the generate form with the original prompt pre-filled. The original image is automatically shown as a reference photo in image-editing mode, allowing the user to nudge the result without starting from scratch.

Regenerate Ad Shot



Product Description *

add one more same product beside

32/300

Background

White marble

Wooden table

Studio white

Tropical

Dark moody

Lighting

Natural window

Studio softbox

Dramatic side

Golden hour

Mood (multiple)

Luxury

Fresh & clean

Minimal

Playful

Angle

Eye-level

45 degrees

Flat lay

Close-up

Props (multiple)

Flowers

Fresh fruits

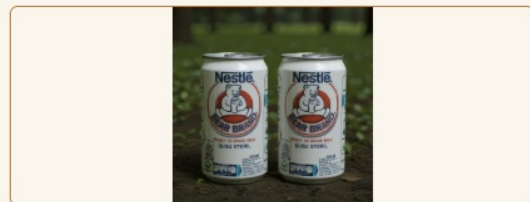
Linen fabric

Water drops

Generated Prompt

Product: add one more same product beside. Commercial product photography, high resolution, sharp focus on product, no watermark, no extra text.

Reference Photo (from original)



Upload Different

JPEG, PNG, WebP — max 5MB

Generate

9 Delete Confirmation

Deleting an ad shot requires explicit confirmation through a modal to prevent accidental data loss. Campaign deletion follows the same pattern and also cascades to all contained ad shots.

Delete Ad Shot



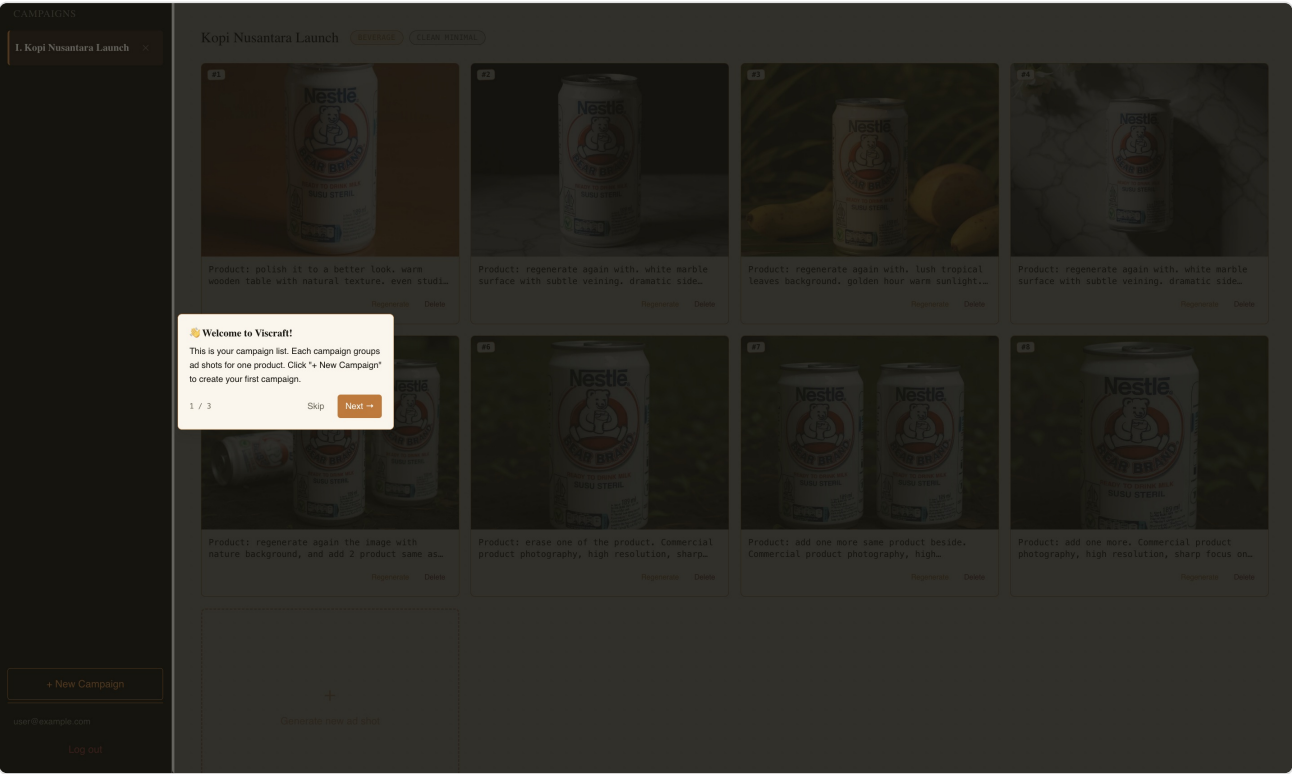
Are you sure you want to delete this ad shot? This action cannot be undone.

Cancel

Delete

10 Onboarding Tour

First-time users see a 3-step guided tour that highlights the campaign sidebar, generate form, and prompt preview — the three core interaction areas of the workspace.



CAMPAIGNS

I. Kopi Nusantara Launch

Kopi Nusantara Launch

1.1



Product: polish it to a better look, warm wooden table with natural texture, even studi...

Regenerate

Delete

1.2



Product: regenerate again with, white marble surface with subtle veining, dramatic side...

Regenerate

Delete

1.3



Product: regenerate again with, lush tropical leaves background, golden hour warm sunlight...

Regenerate

Delete

1.4

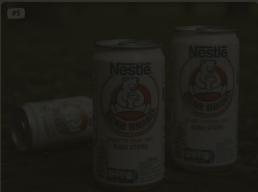


Product: regenerate again with, white marble surface with subtle veining, dramatic side...

Regenerate

Delete

1.5

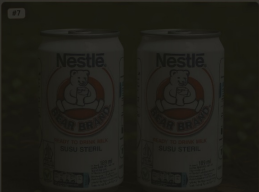


Product: regenerate again the image with nature background, and add 2 product same as...

Regenerate

Delete

1.6



Product: add one more same product beside. Commercial product photography, high...

Regenerate

Delete

+

Generate new ad shot

Generate Ad Shots

Click here to create a new ad shot. You'll describe your product, pick background, lighting, mood, and angle — then the AI builds a professional prompt and generates your ad photo.

2 / 3

Skip

Next →

+ New Campaign

user@example.com

Log out

CAMPAIGNS

I. Kopi Nusantara Launch

Kopi Nusantara Launch

1.1



Product: polish it to a better look, warm wooden table with natural texture, even studi...

Regenerate

Delete

1.2



Product: regenerate again with, white marble surface with subtle veining, dramatic side...

Regenerate

Delete

1.3



Product: regenerate again with, lush tropical leaves background, golden hour warm sunlight...

Regenerate

Delete

1.4

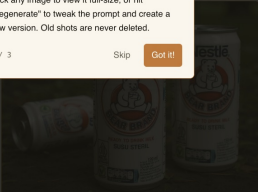


Product: regenerate again with, white marble surface with subtle veining, dramatic side...

Regenerate

Delete

1.5

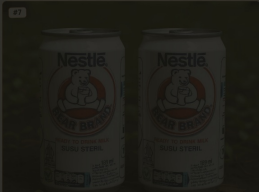


Product: regenerate again the image with nature background, and add 2 product same as...

Regenerate

Delete

1.6



Product: add one more same product beside. Commercial product photography, high...

Regenerate

Delete

+

Generate new ad shot

Regenerate Anytime

After generating, your ad shots appear in the grid. Click any image to view it full-size, or hit "Regenerate" to tweak the prompt and create a new version. Old shots are never deleted.

3 / 3

Skip

Got it!

+ New Campaign

user@example.com

Log out

4. Prompt Building System

Raw product descriptions typed by users ("A bottle of cold brew coffee") produce inconsistent AI results on their own. The `buildPrompt()` function on the frontend assembles a structured prompt that reliably yields commercial-quality product photography.

How `buildPrompt()` Works

The function combines the user's product description with the `promptValue` of each selected styling option, separated into logical sections, and appends a fixed commercial photography suffix.

Example Input

Field	Value
Product Description	A bottle of cold brew coffee
Background	White Marble → white marble surface with subtle veining
Lighting	Natural Window → soft natural window light from left
Mood	Fresh & Clean → fresh clean aesthetic, bright and airy
Angle	45 Degrees → 45-degree angle shot

Assembled Generated Prompt

```
Product: A bottle of cold brew coffee. white marble surface with subtle veining.  
soft natural window light from left. fresh clean aesthetic, bright and airy.  
45-degree angle shot. Commercial product photography, high resolution,  
sharp focus on product, no watermark, no extra text.
```

Storage

Both the raw user input (`user_prompt`) and the assembled prompt (`generated_prompt`) are stored in the database. The `generated_prompt` is what actually gets sent to the Pollinations API. This separation preserves the user's original intent for the regenerate flow while giving the AI enough context to produce quality results.

Prompt Validation

- Minimum length: 3 characters
- Maximum length: 300 characters
- Blocked words: `nude` , `explicit` , `nsfw` , `gore` (case-insensitive)
- Validation runs server-side in `sceneService.validatePrompt()` — the frontend character counter is informational only

5. Error Handling

The application handles three categories of visible failure states, each rendered differently in the UI.

Generation Failure States (Async)

When a generation fails asynchronously, the skeleton polling card resolves into an error card in the gallery. The error card displays the failure reason and an option to retry.

Error Code	Cause	UI Message

ERR_03	Pollinations API did not respond within 60-90 seconds	"Generation timed out. The AI took too long to respond."
ERR_05	Pollinations returned an empty or unparseable response	"Generation failed. The AI returned an unexpected response."
ERR_13	Prompt was rejected by the AI content safety filter	"Content policy violation. Please revise your prompt."

Form Validation Errors (Synchronous)

Validation errors from the server (ERR_04, ERR_11) are shown inline in the generate modal above the form fields. The submit button is also disabled client-side if the prompt is empty.

Rate Limit (HTTP 429)

When the rate limit is hit (ERR_02), the modal displays a dismissable error banner: "Too many requests. Please wait a moment before generating again."

Network / Auth Errors

The Axios JWT interceptor handles 401 responses globally — on token expiry, the user is logged out and redirected to the landing page. Generic network errors are shown via the `ErrorBanner` component at the top of the workspace.

6. Local Setup Instructions

Prerequisites: Go 1.21+, Node.js 18+, PostgreSQL (local or remote)

Backend Setup

```
cd viscraft-backend

# Copy and fill environment variables
cp .env.example .env
```

Edit `.env` with your values:

Variable	Description
DB_HOST	PostgreSQL host (e.g. localhost)
DB_PORT	PostgreSQL port (default: 5432)
DB_NAME	Database name
DB_USER	Database user
DB_PASSWORD	Database password
JWT_SECRET	Random secret string for signing tokens
POLLINATIONS_API_KEY	Your Pollinations.ai API key
APP_PORT	Server port (default: 8089)
STORAGE_PATH	Path for storing generated images (default: ./storage/images)
STORAGE_PUBLIC_URL	Public URL prefix for image files (e.g. http://localhost:8089/storage/images)

```
# Install Go dependencies
go mod download

# Run database migrations in order (001 through 005)
psql -h $DB_HOST -U $DB_USER -d $DB_NAME -f migrations/001_init.sql
psql -h $DB_HOST -U $DB_USER -d $DB_NAME -f migrations/002_add_used_reference_image.sql
psql -h $DB_HOST -U $DB_USER -d $DB_NAME -f migrations/003_scenes.sql
psql -h $DB_HOST -U $DB_USER -d $DB_NAME -f migrations/004_product_ad_pivot.sql
psql -h $DB_HOST -U $DB_USER -d $DB_NAME -f migrations/005_drop_reference_scene_fkey.sql

# Start the server
go run main.go
# Server starts on http://localhost:8089
```

Frontend Setup

```
cd viscraft-frontend

# Install Node dependencies
npm install

# Copy and fill environment variables
cp .env.example .env
```


Edit `.env` :

Variable	Value
VITE_API_BASE_URL	http://localhost:8089

```
# Start the development server
npm run dev
# App runs on http://localhost:5173
```

Running in under 15 minutes: The only hard prerequisites are a running PostgreSQL instance and a Pollinations.ai API key (free tier available at pollinations.ai). Once the migrations are applied and both servers are running, the app is fully functional at `http://localhost:5173` .